FIELD GUIDE · MULTI-AGENT ENGINEERING

Contracts Between Agents

How to build one product with many AI agents, without them breaking each other's work

Sparsh Jain
AI Engineer

Contracts Between Agents
© 2026 Sparsh Jain. All rights reserved.

First edition, October 2026.

This book shares general engineering lessons. All examples are generalized and anonymized; no client, company or product details are disclosed.

You may quote short passages with attribution.

Contents

Preface	The Day Two Agents Disagreed
Chapter 1	The New Team: Many Agents, One Product
Chapter 2	How Agents Break Each Other's Work
Chapter 3	What a Cross-Agent Contract Is
Chapter 4	The Instruction File: Your Team's Constitution
Chapter 5	Ownership and Boundaries
Chapter 6	Writing Good Contracts
Chapter 7	Hand-offs and Integration
Chapter 8	Making Contracts Enforce Themselves
Chapter 9	Case Walkthrough: One Field, Three Spellings
Chapter 10	Changing a Contract Safely
Chapter 11	The Playbook
Appendix	Contract Template and Glossary

PREFACE

The Day Two Agents Disagreed

The bug was tiny. One agent wrote a value as "Female". Another expected "women". A third filled in nothing at all. Every agent had done its own job correctly. The product was still broken.

I was building a multi-part product: a customer app, an admin portal, a set of shared documents and templates. Several AI coding agents worked in parallel. One owned the event-creation form. Another owned the listing screen. A third built filters and reports. Each was fast and capable, and none of them could see what the others had decided.

A single field, describing who an event was meant for, ended up with three different spellings across three parts of the system. Filters silently returned nothing. Reports undercounted. Nothing crashed, which made it worse.

The fix wasn't a smarter model. It was something human engineering teams worked out decades ago: **a contract**. A short, written, shared agreement that said *this field has exactly these three values, this is the default, and every part of the system must use them*. Once that contract existed in a place every agent read, the problem disappeared and stayed gone.

This book is about that idea, scaled up. When several AI agents build one product, they need the same things human teams need: clear ownership, shared definitions, explicit hand-offs and

automatic checks. Agents need them even more, because they are faster than humans, more confident than humans, and they forget everything between sessions.

Who should read this

- **Engineers and tech leads** running more than one coding agent on the same codebase.
- **Founders and product owners** who direct agents to build their products and want fewer mysterious regressions.
- **Anyone curious** about how "agent teams" actually work in practice. The ideas are explained with plain-language diagrams before any code appears.

> A NOTE ON EXAMPLES
>
> All examples are generalized. Field names and scenarios are illustrative, and no project, client or product details are disclosed.

CHAPTER 1

The New Team: Many Agents, One Product

Running several AI agents on one product isn't science fiction any more. It's an everyday workflow, and it comes with a new kind of coordination problem.

Why teams of agents happen

A single agent working on a large product hits limits quickly. Its context fills up, its attention spreads thin, and one long session becomes slow and fragile. The natural answer is to split the work: one agent per feature, per screen, per service, or per role (builder, reviewer, tester). Agents can run in parallel, often in separate working copies of the code, and finish in an afternoon what one agent would take days to do.

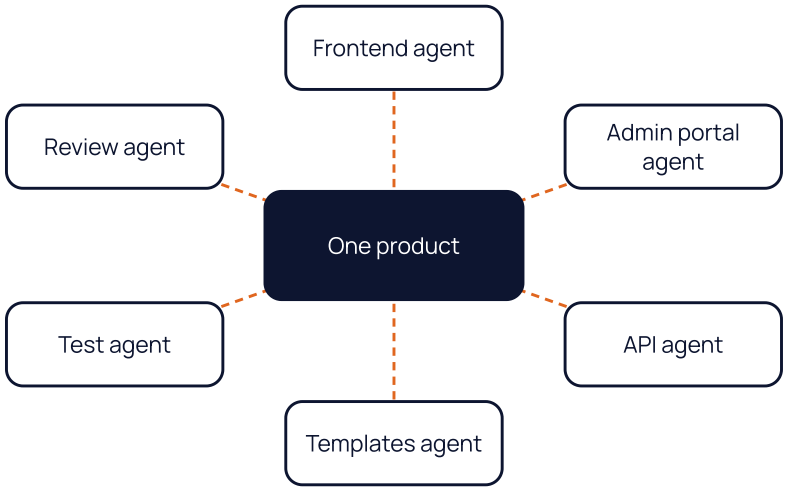


Figure 1.1: A typical agent team. Each agent is strong in its own area and blind to the others by default.

What's different about agent teammates

Trait	Human teammate	Agent teammate
Memory	Remembers last week's decisions	Starts each session fresh unless told
Hallway knowledge	Overhears, asks, notices	Knows only what's in its context
Speed	Hours per feature	Minutes per feature
Confidence	Says "not sure, let me check"	Often fills gaps with plausible guesses
Consistency	Habits drift slowly	Follows written rules very consistently

That last row is the opportunity. Agents are bad at *implicit* agreements and very good at *explicit* ones. Write the agreement down where they'll read it, and they follow it better than most humans do.

The coordination problem in one picture



Figure 1.2: How agent teams drift apart. Nobody did anything wrong, and the product still broke.

Every chapter that follows is a way to break this chain at the second box: make sure decisions that affect more than one agent get written down, in a form other agents can find, read and be checked against.

CHAPTER 2

How Agents Break Each Other's Work

Before designing contracts, you need to recognize the failures they prevent. These are the six I saw most often.

---

1. Vocabulary drift

The same concept gets different names or values: *female / women / F*, *cancelled / canceled*, *startDate / start_date / eventStart*. Each choice is reasonable on its own, and together they're incompatible. This was the most common failure by far.

2. Shape drift

One agent stores a price as a number in rupees, another as a string with a currency symbol, a third in the smallest unit. One stores a date as a local date, another as a timestamp. Data flows, but meaning doesn't.

3. Silent defaults

When a value is missing, what should happen? One agent treats "missing" as "everyone", another as "nobody", a third throws an error. Unstated defaults are the most dangerous form of drift, because they only show up on edge cases.

4. Overlapping edits

Two agents edit the same file or component at once. One "cleans up" what the other just built. Without clear ownership, parallel agents can undo each other in minutes.

5. Ghost decisions

A decision is made in one agent's conversation ("let's make that field required") and dies with that session. The next agent, or the same agent tomorrow, never learns about it.

6. Confident integration

An agent integrates with another part of the system by *assuming* how it works rather than reading it. The code looks right, compiles, and calls an endpoint or field that doesn't exist.

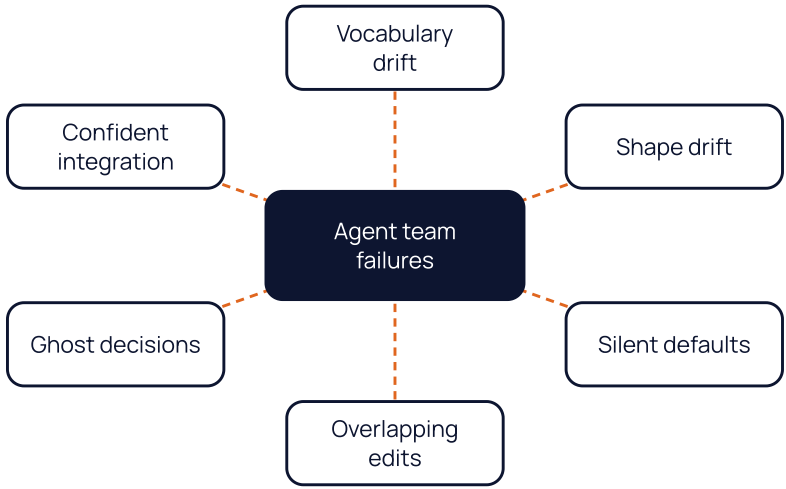


Figure 2.1: The six failure modes. All six come down to a decision that wasn't shared.

THE COMMON ROOT

Every failure above has the same cause: a decision that affects more than one agent lives in only one agent's head. The cure is the same each time too: move that decision into a shared, written, checkable place.

CHAPTER 3

What a Cross-Agent Contract Is

A cross-agent contract is a small, explicit, shared agreement about something more than one agent touches. It's short enough to read in a minute and strict enough to be checked by a machine.

The definition

A cross-agent contract states, for one shared concept: its name, its exact allowed values or shape, its default, who owns it, and how it is checked.

That's all. A contract isn't a design document or a specification of a whole feature. It's the narrow interface where different agents' work meets.

Anatomy of a contract

```
CONTRACT: event.audience
Owner:     Event data (admin portal agent)
Used by:   Event form, listing screen, filters, reports
Values:    EXACTLY one of: "women" | "men" | "anyone"
Default:   "anyone"  (when missing, null or unknown)
Display:   "Women only" | "Men only" | "Open to all"
Rules:     Store the value, never the display label.
           Never invent new values; propose a contract change.
Checked by: shared type + validation + test (see §8)
Changed:   2026-10 — initial agreement
```

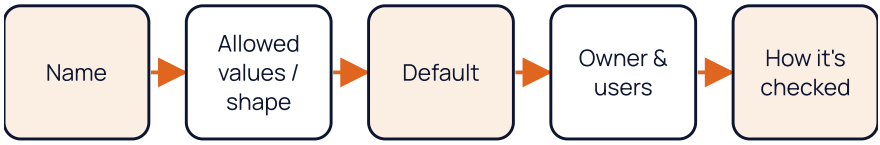


Figure 3.1: The five parts of every contract.

Why each part matters

Name

One canonical name, with its exact path in the data. Aliases are where vocabulary drift starts.

Allowed values or shape

The word **"exactly"** is doing a lot of work. A closed list ("exactly these three") can be enforced. An open description ("a gender-like value") can't.

Default

Say what "missing" means. In the opening story, choosing *"anyone"* as the default meant old records with no value kept showing up in listings instead of vanishing.

Owner and users

The owner is the only agent (or person) allowed to change the contract. Users must follow it. Listing the users means you know who to notify when it changes.

How it's checked

A contract no machine checks is a suggestion. Chapter 8 shows how to make every contract enforce itself.

Store the value, display the label

One rule deserves special attention: **separate the stored value from the displayed text.** Agents love to "improve" labels: "Women only" becomes "Ladies" in one screen and "Female travellers" in another. That's fine for display, and disastrous if those labels leak into stored data. The contract fixes the stored value, while display labels can be adjusted freely in one mapping.

CHAPTER 4

The Instruction File: Your Team's Constitution

Agents read before they act, but only what you put in front of them. The project instruction file is the one document every agent reads at the start of every session.

What it is

Most modern coding agents automatically load a plain-text instruction file from the root of a project, often named something like *AGENTS.md* or a tool-specific equivalent. Whatever is in that file becomes part of every session's starting context. For an agent team, it's the closest thing you have to a shared memory and a shared rulebook.



Figure 4.1: The session ritual. The first two steps are what keep agents aligned.

What belongs in it

Section	Contents
Product in brief	What the product is and who it serves, in five lines

Map of the code	Main folders and what lives where
Ownership	Which agent or person owns which area (Chapter 5)
Contracts index	A list of every contract with a one-line summary and a link
Conventions	Naming, formatting, error handling, dates, money
How to verify	Commands to build, test and check contracts
Decision log	Short dated entries of choices that affect everyone
Never do	Red lines: e.g. never edit another area's files, never change a contract unilaterally

Keep it short, link out

A ten-page instruction file gets skimmed by agents just as it does by humans, and it eats context on every session. Keep the root file to one or two pages. Put each contract in its own small file, and let the root file index them. Agents then load the detail only when they touch that area.

```
/AGENTS.md                ← 1–2 pages, read every session
/contracts/
    event-audience.md     ← one contract per file
    money.md
    dates-and-timezones.md
    booking-status.md
/docs/decisions.md        ← dated decision log
```

LESSON LEARNED

Early on, I let each agent keep its own notes. Within a week there were four slightly different versions of the truth. Moving to one root file plus one-contract-per-file ended that. The rule is simple: if it affects more than one agent, it lives in the shared files, nowhere else.

The decision log

Ghost decisions (Chapter 2) die here. Whenever an agent or a human makes a choice that other parts of the system depend on, it gets one dated line in the log: what was decided, why, and which contract it touches. Ask every agent to end its session by adding to the log if it made such a decision. It takes seconds and saves hours.

```
2026-10-12  Prices stored as integer minor units. (contract: money)
2026-10-19  Event audience limited to 3 values, default "anyone".
            (contract: event-audience) Replaces free-text field.
2026-10-23  All dates stored in UTC; displayed in local time.
```

CHAPTER 5

Ownership and Boundaries

Parallel agents are only safe when each one knows exactly where its territory ends. Ownership turns "who changed this?" into a question you never have to ask.

Draw the map

Before you start parallel work, split the product into areas with clear edges: by app (customer app, admin portal), by layer (interface, API, data), or by feature. Assign each area exactly one owner. Shared territory, such as common components, types and contracts, gets its own owner too, often you.

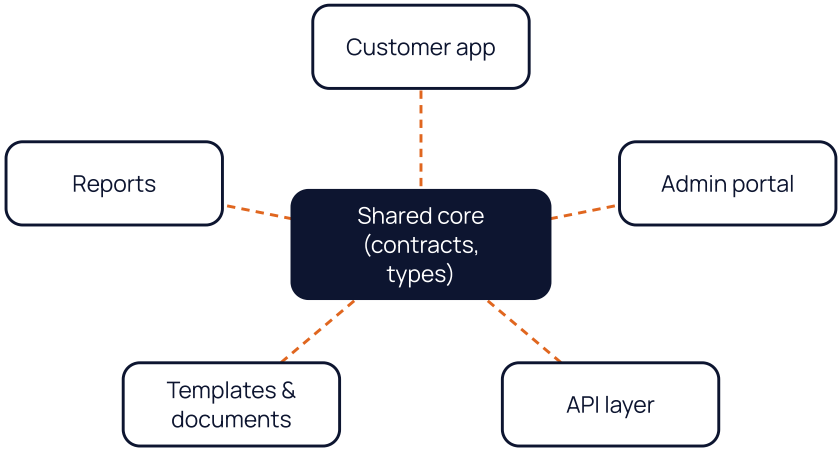


Figure 5.1: Areas with single owners around a shared core. Only the core's owner changes the core.

Three rules of territory

1. **Write only in your area.** An agent may read anything but changes only files in its own area.
2. **Request, don't reach.** If you need a change in someone else's area, write a request (a note, an issue, a message to the owner) instead of editing it yourself.
3. **The shared core changes by proposal.** Contracts and shared types change only through the owner, with all users informed.

Physical isolation helps

Many agent tools can give each agent its own isolated working copy of the code, on its own branch. That makes overlapping edits impossible during work and pushes conflicts to a clear, reviewable moment: the merge. Combine isolation with ownership and the most chaotic failure mode, agents overwriting each other, mostly disappears.



Figure 5.2: Isolated parallel work with a single integration point.

PRACTICAL TIP

Put the ownership map in the instruction file as a simple table: area, folders, owner. Agents follow it remarkably well once it's written down, and tell you when a task requires stepping outside their area.

CHAPTER 6

Writing Good Contracts

Most contracts are small. The skill is knowing what to put in them, what to leave out, and how to phrase them so neither a human nor an agent can misread them.

What deserves a contract?

Write a contract for anything that is **shared** and **easy to get subtly wrong**:

- **Enumerations:** statuses, categories, types, audiences, roles.
- **Money:** unit, currency, rounding, tax inclusion.
- **Time:** timezone, format, what "today" means, inclusive or exclusive ranges.
- **Identifiers:** format, who generates them, whether they're ever reused.
- **State machines:** allowed statuses and allowed transitions between them.
- **Interfaces:** request and response shapes between parts of the system.
- **Files and naming:** where outputs go and what they're called.

Phrasing rules

Instead of…	Write…
"Use a gender value"	"EXACTLY one of: women | men | anyone"

"Store the price"	"Integer, minor currency units, tax-exclusive"
"Handle missing values sensibly"	"If missing or null → treat as anyone"
"Use the right status"	"draft → published → archived; no other transitions"
"Dates in the usual format"	"ISO 8601, UTC, stored; local only for display"

Capital-letter words like EXACTLY, NEVER and ONLY aren't shouting. They're anchors. Agents give them weight, and human reviewers spot them instantly.

A state-machine contract

```
CONTRACT: booking.status
Values:      enquiry | confirmed | in_progress | completed |
cancelled
Default:     enquiry
Transitions: enquiry → confirmed | cancelled
             confirmed → in_progress | cancelled
             in_progress → completed
             (no other transitions; completed and cancelled are
final)
Rule:        Only the bookings area may change status.
```

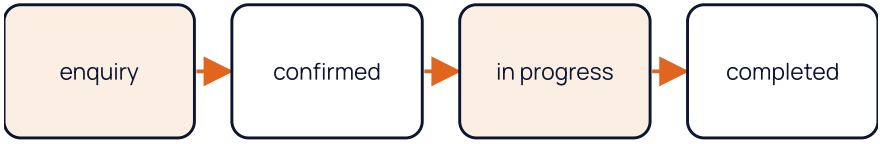


Figure 6.1: A state machine is a contract about movement, not just values. Cancelled is reachable from the first two states only.

Keep contracts small and boring

A contract that tries to describe a whole feature will be ignored or misread. One concept per contract, one screen of text at most. If you find yourself writing paragraphs of reasoning, put the reasoning in the decision log and keep only the rule in the contract.

> **INCLUDE EXAMPLES OF WHAT'S WRONG**
>
> Agents learn as much from counter-examples as from rules. One line such as *Wrong: "Female", "Ladies", "" (empty string)* under an enumeration contract prevents the most likely mistakes before they happen.

CHAPTER 7

Hand-offs and Integration

Agents work in parallel, but products are used as a whole. The moments where one agent's work meets another's are where contracts earn their keep.

The hand-off note

When an agent finishes a piece of work that others depend on, it should leave a short hand-off note. Not a long report, just the facts the next agent needs.

```
HAND-OFF · Event form (admin portal)
Done:      Form saves event.audience using contract values.
Contracts: event-audience (followed), dates (followed)
Changed:   Added field event.capacity (integer, optional)
           → proposed new contract: event-capacity.md
Needs:     Listing screen should show capacity when present.
Verify:    run "check-contracts" and "test events"
```

Contract-first integration

The safest order for building connected features is: **contract first, then both sides.** Before the form agent and the listing agent start, the shared contract for the data between them is written and agreed. Both agents then build against the same written truth, in parallel, without needing to see each other's code.

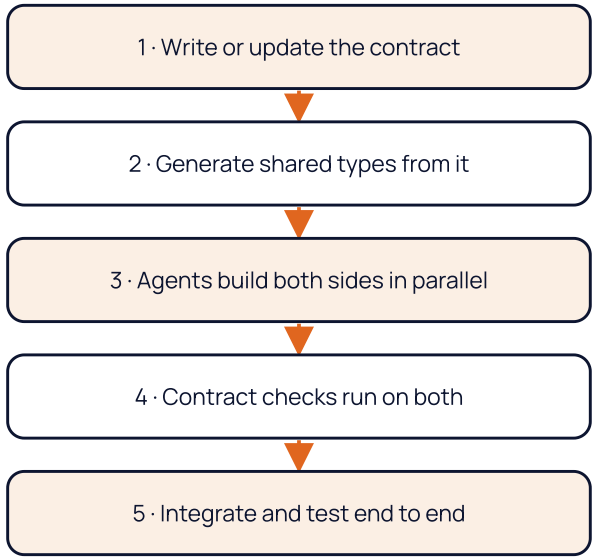


Figure 7.1: Contract-first integration lets parallel agents meet in the middle without surprises.

Read, don't assume

The "confident integration" failure from Chapter 2 has a simple remedy built into every agent's brief: *before using any part of the system you don't own, read its contract or its code, and never assume a name, field or endpoint exists.* It sounds obvious. Writing it down measurably reduced invented references in my projects.

A coordinating agent

For larger efforts, one agent (or a person) can play the coordinator role. It breaks work into tasks per area, makes sure the contracts exist first, hands tasks to specialist agents, collects their hand-off notes, and runs the integration checks. The coordinator doesn't write feature code. It protects the seams.

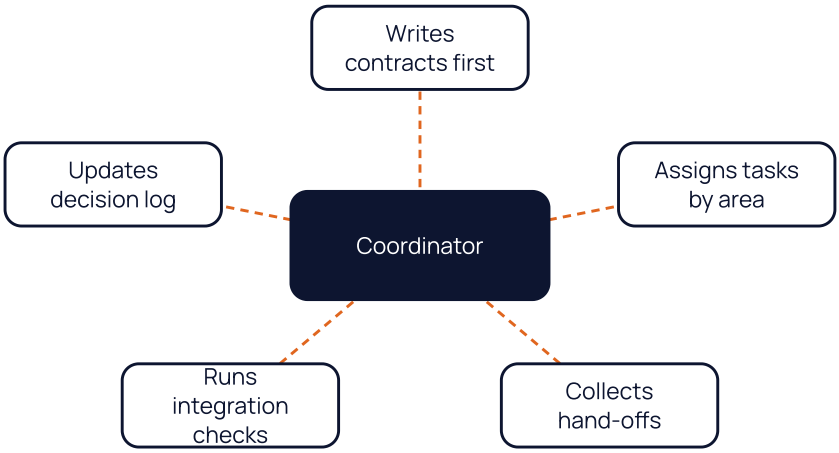


Figure 7.2: The coordinator's job is the seams, not the features.

CHAPTER 8

Making Contracts Enforce Themselves

A written contract prevents most drift. An automatically checked contract prevents nearly all of it, including drift you would never have noticed.

Four layers of enforcement

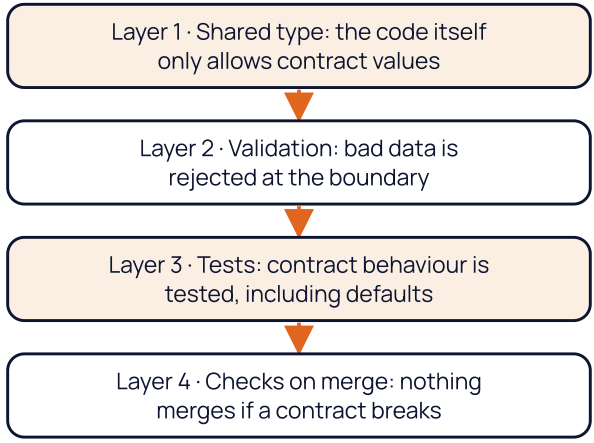


Figure 8.1: Each layer catches what the one above missed.

Layer 1: Shared types

Turn every enumeration contract into a single shared type or constant that all areas import. When an agent tries to write "Female", the code doesn't compile. This is the cheapest and most powerful layer.

```
// contracts/eventAudience.ts — the only definition, imported
everywhere
export const AUDIENCE = ["women", "men", "anyone"] as const;
export type Audience = typeof AUDIENCE[number];
export const DEFAULT_AUDIENCE: Audience = "anyone";

export function normalizeAudience(v: unknown): Audience {
  return AUDIENCE.includes(v as Audience) ? (v as Audience)
                                          : DEFAULT_AUDIENCE;
}
```

Layer 2: Validation at the boundary

Where data enters the system (forms, APIs, imports), validate against the contract and reject or normalize anything outside it. Old data and external sources are where contract violations hide.

Layer 3: Contract tests

Write small tests that state the contract as behaviour: only these values are accepted, missing becomes the default, the display mapping covers every value, and the state machine refuses illegal transitions. Agents are very good at writing these tests once they've seen the contract file.

```
test("audience defaults to anyone", () => {
  expect(normalizeAudience(undefined)).toBe("anyone");
  expect(normalizeAudience("Female")).toBe("anyone");
});
test("every audience value has a display label", () => {
  for (const a of AUDIENCE) expect(LABELS[a]).toBeTruthy();
});
```

Layer 4: Checks on merge

Run the build, type checks and contract tests automatically before any agent's work is merged. If a contract breaks, the merge stops. The agent gets a precise error, and in my experience it usually fixes the problem on its own within one attempt.

> A SIMPLE SWEEP CHECK
>
> A small script that searches the codebase for raw string values that should come from a contract, such as a stray "female" or "Cancelled", catches drift that types miss, especially in templates, queries and configuration files.

Migrating old data

When a contract is introduced after data already exists, as in the opening story, write a one-time migration that maps every old variant to the new values ("Female", "Ladies", "F" → "women"; empty → "anyone"). Then add validation so old variants can't return. Record the migration in the decision log.

CHAPTER 9

Case Walkthrough: One Field, Three Spellings

Here is the opening story, generalized, step by step: what broke, how it was found, and how the contract approach fixed it for good.

The setup

A product had events that could be intended for a specific audience. Three agents worked in parallel. One built the event-creation form in an admin portal, one built the public listing with filters, and one built operator reports.

What happened

Area	Value written or expected	Effect
Event form	Saved the display label, e.g. "Female"	Data stored as labels
Listing filters	Filtered on "women"	Filtered events never appeared
Reports	Treated missing as excluded	Older events undercounted

No errors and no crashes. The symptom reported was vague ("the filter seems broken sometimes"), which is exactly how contract drift usually shows up.

Diagnosis

A search for every place the field was read or written showed three vocabularies and two different assumptions about missing values. The root cause wasn't a bug in any single area. It was the *absence of an agreement* between them.

The fix, in order

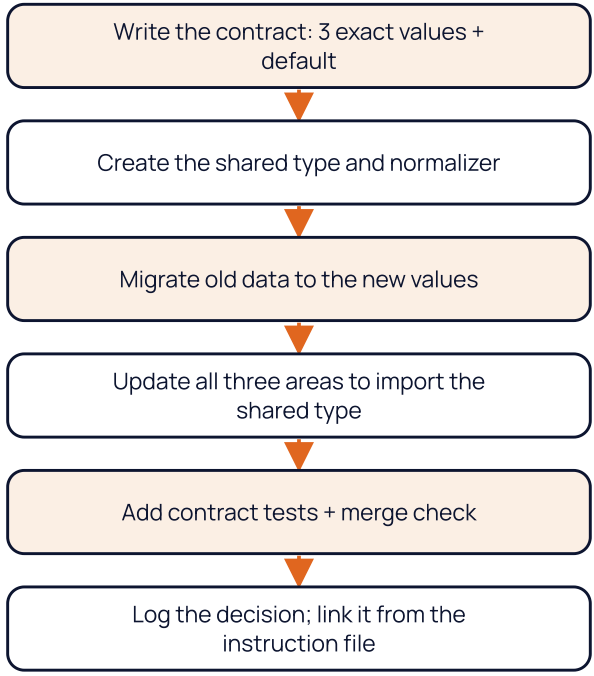


Figure 9.1: The six-step fix. Steps 2 and 5 are what stop it from ever returning.

The outcome

- Filters and reports agreed immediately after migration.

- Later agents working on new screens used the shared type without being told. The instruction file pointed them to it.
- An attempt weeks later to add a fourth value went through a contract change instead of quietly slipping in.

> **THE GENERAL LESSON**
>
> When agents disagree, don't fix the symptom in whichever area is broken. Find the shared concept, write the contract, make it enforceable, and point every agent at it. Fixing one area just moves the disagreement somewhere else.

CHAPTER 10

Changing a Contract Safely

Contracts aren't frozen. Products evolve and contracts must follow, but through a visible process rather than a quiet edit.

The change process



Figure 10.1: A contract change touches every user at once, never one at a time.

Additive vs breaking changes

Change	Type	What's needed
Add a new optional field	Additive	Update contract and type; users may ignore it
Add a new enumeration value	Usually breaking	Every user must handle it (filters, labels, reports)
Rename a value or field	Breaking	Migration + update all users in one change
Change a default	Breaking, often silently	Explicit review of every place "missing" occurs

Adding a value to an enumeration feels harmless and is one of the most common sources of silent breakage. A display mapping without the new label, or a filter that doesn't know it, will quietly misbehave. Contract tests like "every value has a label" catch exactly this.

Let agents propose, not decide

Agents will regularly run into a contract that doesn't fit what they're building. That's healthy. The rule in their brief should be: *if a contract blocks you, stop and write a proposal; do not work around it.* In practice agents follow this well, and the proposals are often good. They are, after all, the ones closest to the code.

CHAPTER 11

The Playbook

Everything in this book, as a sequence you can follow on your next multi-agent project.

Before agents start

- ☐ Write a 1–2 page instruction file at the project root.
- ☐ Draw the ownership map: areas, folders, owners.
- ☐ List the shared concepts and write a contract for each risky one.
- ☐ Turn enumeration contracts into shared types.
- ☐ Create the decision log.
- ☐ Set up automatic checks: build, types, contract tests.

During work

- ☐ One agent per area, ideally in an isolated working copy.
- ☐ Contract first for any feature that crosses areas.
- ☐ Agents read contracts before using anything they don't own.
- ☐ Requests across areas go to owners, not direct edits.
- ☐ Every session ends with a hand-off note and decision-log entries.

At integration

- ☐ All checks pass before merge.

- ☐ Run a sweep for raw values that bypass contracts.
- ☐ Test end to end across the areas that meet.
- ☐ Update the contracts index if anything changed.

Agent brief snippet

```
## Working in this project
- Read /AGENTS.md first, then every contract for data you touch.
- Edit ONLY files in your area: [area, folders].
- Use shared types from /contracts. NEVER hard-code their values.
- If a contract blocks you: STOP and write a proposal in
  /docs/proposals/. Do not work around it.
- Never assume a field, endpoint or file exists. Read it first.
- Before finishing: run checks, write a hand-off note, add any
  cross-area decision to /docs/decisions.md.
```

"Agents don't need more intelligence to work together. They need the same thing people do: shared, written agreements."

APPENDIX

Contract Template and Glossary

Contract template

```
CONTRACT: [domain.concept]
Owner:      [area or person]
Used by:    [list of areas]
Values / shape:
            [EXACTLY ... | type and units | structure]
Default:    [what missing / null / unknown means]
Display:    [labels, if any; never stored]
Rules:      [store vs display; who may change; transitions]
Wrong:      [common incorrect values or shapes]
Checked by: [shared type | validation | tests | merge check]
History:    [date — change — reason]
```

Glossary

Term	Meaning
Cross-agent contract	A small, explicit agreement about a concept several agents touch.
Instruction file	The project-level document every agent reads at session start.
Shared type	A single code definition of a contract, imported by every area.
Drift	Gradual divergence in names, values or assumptions between agents.

Ownership map	The table of areas, folders and their single owner.
Hand-off note	A short summary an agent leaves for those who depend on its work.
Decision log	Dated, one-line record of choices that affect more than one area.
Coordinator	An agent or person who protects the seams between areas.
Migration	A one-time change that brings old data in line with a contract.

About the Author

Sparsh Jain is an AI engineer who builds software products with teams of AI coding agents working side by side, across web apps, admin portals, document systems and automation pipelines.

His work focuses on the coordination problem: how to keep many fast, capable, forgetful agents aligned on one product. This book collects the patterns that held up in real projects.

If it saved you one broken build, pass it to the next engineer who says "just let the agents figure it out".